# Christopher Okoro, IU Username: cookoro
# Reddit Data Ingestion and Transformation Pipeline – Final Project

## 1. Introduction

According to Datacamp, Data is the lifeblood of modern businesses and organizations, driving decision-making, insights, and innovation. Modern data-intensive systems rely on well-designed pipelines to ensure that information flows reliably from raw ingestion to final, usable formats. Building a reliable, secure, and timely pipeline is a crucial part of data-driven decision-making. In this project, I focused on the **Lifecycle and Pipelines module** by designing and implementing a multi-stage data processing pipeline using Google Cloud Dataflow and Apache Beam. The pipeline ingests weekly Reddit data stored in Google Cloud Storage (buckets), processes each batch through structured validation and transformation steps, and organizes the output into several lifecycle-appropriate storage layers. This design reflects the essential concepts of data pipelines taught in our course, including staged processing, schema enforcement, data cleaning, and reproducibility.

The purpose of this work was to implement a cloud-based ETL workflow that automatically separates raw input into clean, filtered, invalid, and logged datasets. The system reads CSV files, parses, and validates each record, applies a rule that selects only afternoon posts (between 12:00-17:00) with more than five comments, and writes results to corresponding storage tiers. In one sentence, this project implements a complete end-to-end cloud pipeline that demonstrates the lifecycle of data as it moves from raw ingestion to validated, curated, and quarantined outputs.

## 2. Background

Working with large and continuously growing datasets requires more than simple scripts and cleaning steps; it requires a structured lifecycle that manages how data is ingested, validated, transformed, and stored. The Lifecycle and Pipelines module emphasized the importance of organizing data into stages, such as raw, clean, filtered, and quarantine. This is done to maintain reliability and traceability as data moves through a system. This project applies two key lifecycle concepts from the course:

- staged data processing, where different versions of the data exist simultaneously but serve different quality purposes, and
- data validation and schema enforcement, which ensure that downstream consumers receive consistent, trustworthy data.

In addition to lifecycle concepts, this project also incorporates ideas from other parts of the course, specifically Cloud Computing and Big Data Processing. Cloud computing concepts guided the use of managed services like GCS (Google Cloud Storage) and Google Cloud Dataflow, which offer scalability, resource abstraction, and reproducibility. Big data processing principles informed the design of a pipeline capable of handling large semi-structured datasets, performing distributed parsing and transformation, and writing results

efficiently to multiple storage layers. In my project, a small data sample was used, but ideally pipelines are optimized for the 3 V's of data. Together, these concepts demonstrate why well-designed pipelines are essential in modern data ecosystems: they provide the structure, automation, and scalability required to manage complex data flows in a cloud environment.

## 3. Methodology

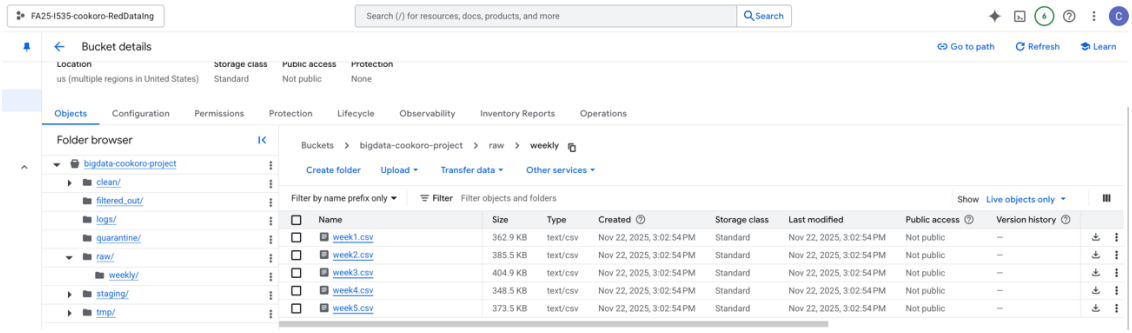
**Environment**
This project was implemented using Google Cloud Platform (GCP), which provided a managed and reproducible environment for building and executing the data pipeline. The key services used were:

- Google Cloud Storage (GCS) – GCS buckets served as the main storage layer for raw, clean, filtered, quarantined, and logged data.
- Google Cloud Dataflow – executed the pipeline using the Apache Beam SDK and handled distributed processing, autoscaling, and orchestration.
- Apache Beam SDK (Python) – provided the programming model for reading, transforming, validating, and writing data.
- Local development environment (PyCharm + virtual environment) – used for authoring the pipeline code and submitting jobs.

GCP was selected because it aligned with course concepts on cloud computing and because Dataflow integrates natively with Beam, allowing the pipeline to scale and run reliably without managing underlying infrastructure.

The versions/structure used are below:

- Python 3.10 in my PyCharm IDE (any python IDE can be used to reproduce). This version of python is sable for the libraries used.
- Apache Beam 2.69.0 (with the gcp extra)
- Dataflow Runner for pipeline execution
- Google Cloud SDK 497.0.0
- GCS buckets with the following folder structure shown below:



**Pipeline Design and Workflow**

The pipeline follows a stepwise ETL workflow using Apache Beam transforms:

1. Read Input Files: The pipeline reads one weekly CSV file at a time from the raw/weekly/ directory on Cloud Storage Bucket.
2. Parse CSV Lines: Each line is split into structured fields. Missing fields or malformed rows are flagged early for quarantine.
3. Drop Rows With None / Empty Required Fields: Records missing essential fields such as timestamp, post text, or comment count are routed directly to the quarantine output.
4. Validation and Schema Enforcement: Data Types were checked, “created_utc” is converted from unix timestamp to ISO datetime, and validation errors are logged for traceability.
5. Afternoon Criteria Filter: The rule of accepting only posts created between 12:00 and 17:00 (afternoon) AND with more than 5 comments is applied.
6. Classification of Invalid Rows: Rows that fail are quarantined and added to the quarantine csv and the failure is logged in the logs/  json file.
7. The pipeline writes the results into sharded output files which will be shown in the result section.

**Architecture Diagram**

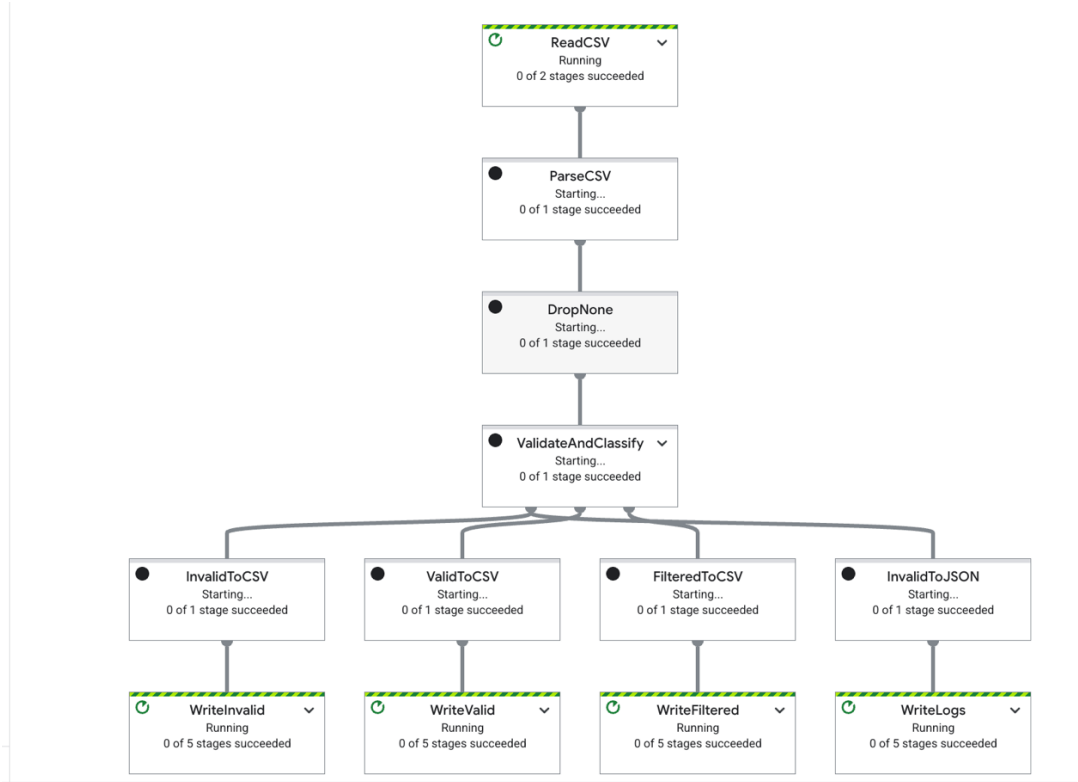



Figure 3.1 Job Graph

Above is my generated Job Graph, it explains the architecture of my pipeline, and adds visualization to the pipeline design and workflow sections. The diagram shows how each input row takes one of three paths depending on quality and set criteria.

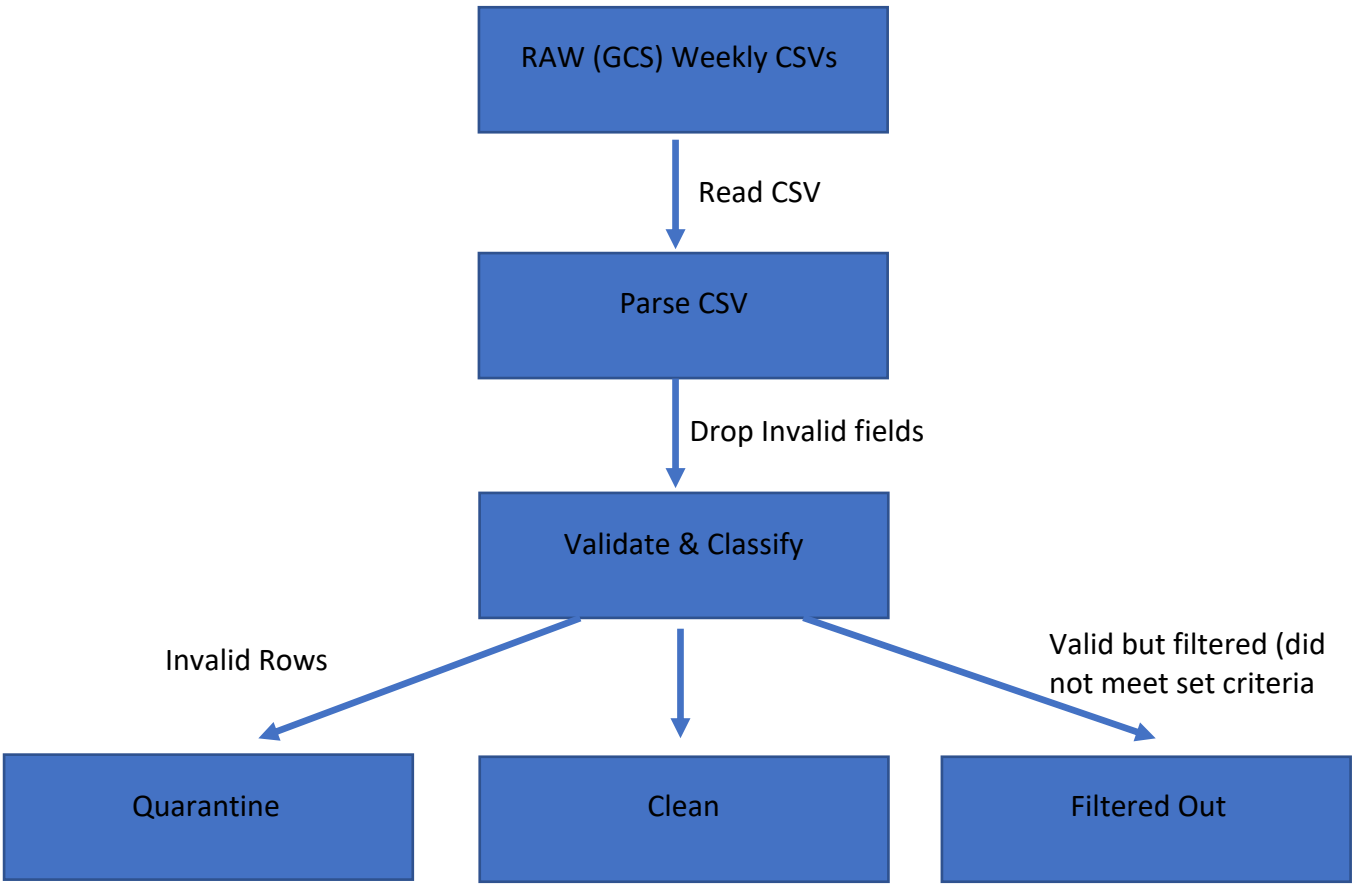



Figure 3.2 Architecture Workflow Diagram

**Data Description**

The dataset consists of 1000 Reddit posts spread across five weekly CSV files. Each item includes attributes such as, id, subreddit, created_utc, title, selftext, score, num_comments, upvote_ratio, title_andd_text, url.

**Validation Plan**

To ensure the pipeline produced correct and reliable outputs, the following validation checks were implemented:

| Validation Check | Purpose | Outcome |
|---|---|---|
| Schema validation | Ensures mandatory fields exist | Missing → quarantine |
| Timestamp conversion | Converts Unix → datetime for filtering | Failed conversion → quarantine |
| Data type checks | Casts numeric fields | Errors logged and quarantined |
| Business rule filter | Afternoon posts + >5 comments | Passed → clean; failed → filtered_out |
| Output integrity | Confirms all outputs written to correct folders | Verified manually in GCS |
| Run consistency | Running weeks 1–5 produces parallel structured outputs | Confirmed |

Table 3.1 Validation Plan

This validation plan demonstrates that the pipeline was not only functional but reliable and reproducible across all five ingestion cycles.

## 4. Results

This section presents evidence that the pipeline operated as intended across the five weekly ingestion cycles. The results include screenshots of the cloud environment, Dataflow execution graphs, and the output files written to each lifecycle storage tier. Together, these confirm that the pipeline correctly ingested, validated, transformed, and classified the Reddit data.

**Cloud Storage Setup**
Before running the pipeline, a Google Cloud Storage bucket was created to organize all lifecycle stages of the data. The folder structure includes dedicated areas for raw data, clean outputs, filtered records, quarantine files, logs, staging, and temporary processing.

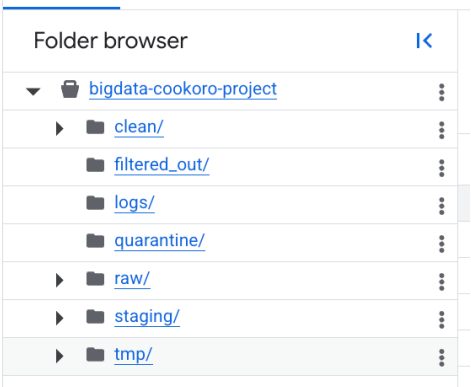



Figure 4.1 GCS Bucket folder structure

The raw/weekly/ folder contains the original dataset for all five weeks:

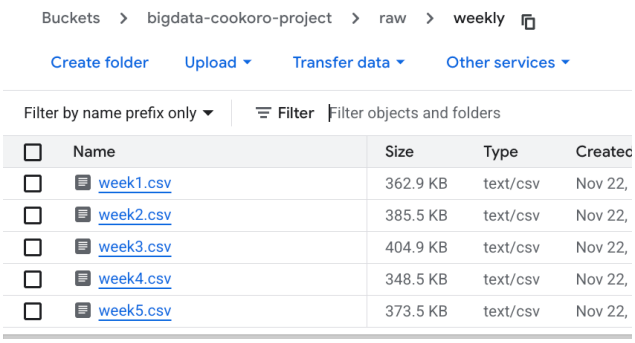



Figure 4.2 Raw CSVs

This folder structure ensured clear separation between ingestion, validation, and processing stages.

**Network and Runtime Configuration**

To allow Dataflow jobs to execute successfully, a custom VPC was created along with the necessary firewall rules and service account permissions.

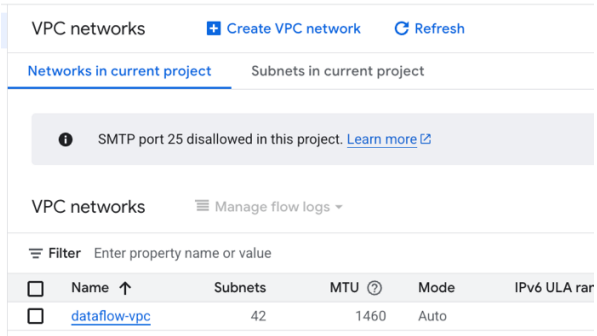



Figure 4.3 VPC network

I had initial errors while setting up as apache beam could not connect to the bucket. After authenticating, creating VPC network, and setting the VPC firewalls, this allowed all Dataflow jobs to initialize worker nodes and access Cloud Storage securely.

VPC firewall rules  Create VPC firewall rule  Refresh

Filter  Enter property name or value

| Name | Enforcement order | Deployment scope | Priority | Description | Direction of traffic | Target | Source | Destination | Protocols and ports | Action | Hit count | Last hit |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| dataflow-vpc-allow-ssh | 1 | Global | 65534 | Allows TCP connections from any source to any instance on the network using port 22. | Ingress | Appl... | IPv4... | – | tcp:22 | Allow | – | – |
| dataflow-vpc-allow-custom | 1 | Global | 65534 | Allows connection from any source to any instance on the network using custom protocols. | Ingress | Appl... | IPv4... | – | All | Allow | – | – |
| dataflow-vpc-allow-rdp | 1 | Global | 65534 | Allows RDP connections from any source to any instance on the network using port 3389. | Ingress | Appl... | IPv4... | – | tcp:3389 | Allow | – | – |
| dataflow-vpc-allow-icmp | 1 | Global | 65534 | Allows ICMP connections from any source to any instance on the network. | Ingress | Appl... | IPv4... | – | icmp | Allow | – | – |

Figure 4.4 VPC Firewall Rules

## Apache Beam Python script

The code below was used with the apache beam pipeline.py, where all the transformations were defined and uploaded to the dataflow worker.

```
(.venv) chrisokoro@Mac reddit-bigdata-project % python3 pipeline.py \
  --runner=DataflowRunner \
  --project=fa25-i535-cookoro-reddataing \
  --region=us-central1 \
  --network=dataflow-vpc \
  --temp_location=gs://bigdata-cookoro-project/tmp/ \
  --staging_location=gs://bigdata-cookoro-project/staging/ \
  --input=gs://bigdata-cookoro-project/raw/weekly/week5.csv \
  --output_clean=gs://bigdata-cookoro-project/clean/week5_valid \
  --output_quarantine=gs://bigdata-cookoro-project/quarantine/week5_invalid \
  --output_filtered=gs://bigdata-cookoro-project/filtered_out/week5_filtered \
  --log_output=gs://bigdata-cookoro-project/logs/week5_invalid_log \
  --requirements_file=requirements.txt
```

Figure 4.5 Python code to run pipeline

## Dataflow Pipeline Execution

The Dataflow job graph provides visual confirmation that the pipeline executed all stages of the ETL workflow. The graph includes all transforms defined in the Apache Beam script—reading the CSV, parsing, dropping missing values, validating, classifying, and writing to each output category.

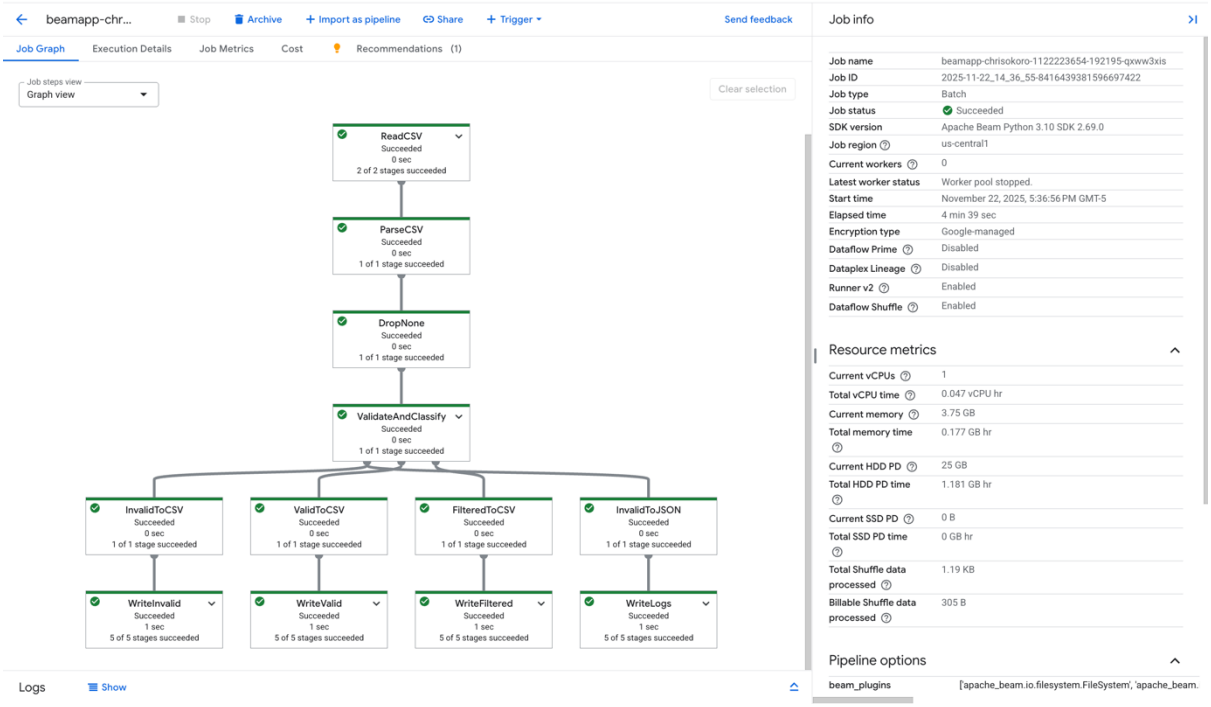



Figure 4.6 Dataflow DAG with Job info

Once the job completed, each stage reported 100% success, and this was executed for the remaining 4 CSVs. This is shown below:

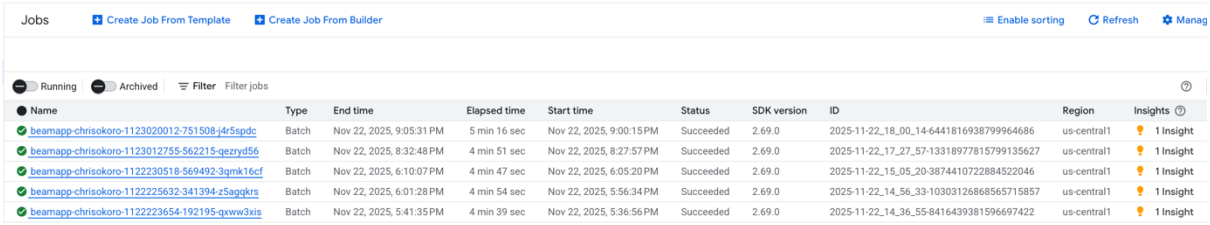



Figure 4.7 Successful Dataflow jobs

## Output Lifecycle Tiers

For each ingestion cycle, the pipeline generated four distinct outputs:

- Clean output: Records that passed all schema checks and met the afternoon and comment threshold were written to the clean directory.

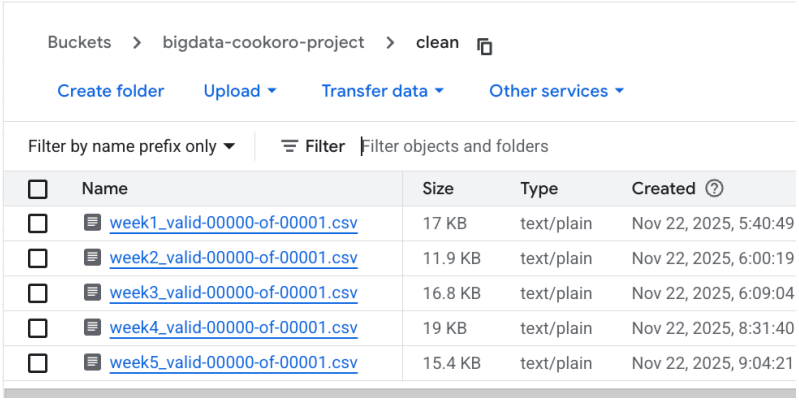



Figure 4.8 GCS Bucket clean directory

- Filtered Output: Rows that were valid but did not satisfy the business rule were routed to the filtered_out directory

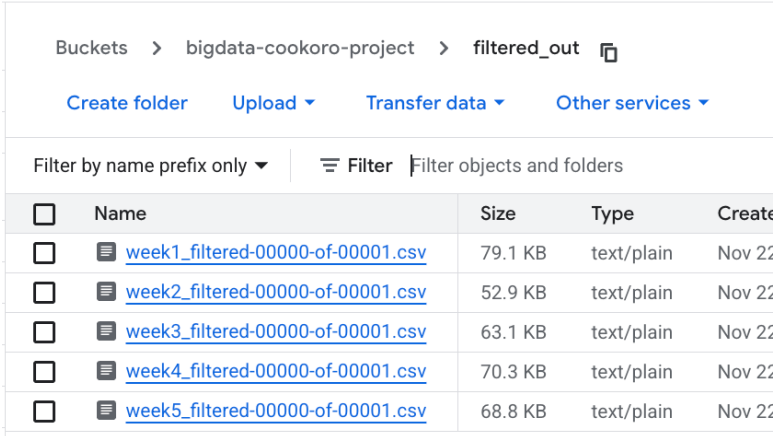



Figure 4.9 GCS Bucket filtered out directory

- Quarantine output: Rows that failed parsing, timestamp conversion, or mandatory field checks were stored in quarantine:

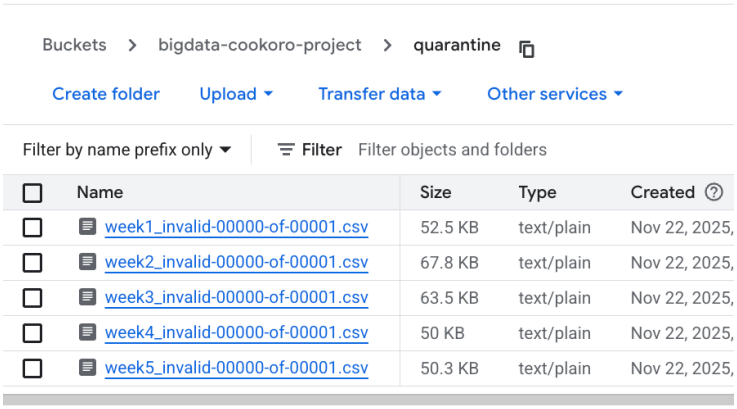



Figure 4.10 GCS Bucket quarantine directory

- Error Logs: Detailed error logs were generated for each invalid row, stored as JSON files. These logs are created to capture information for debugging and auditing.

Below is a screenshot of the transformed, clean data points

| | A | B | C | D | E | F | G | H | I | J | K | L | M |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | id | subreddit | created_utc | title | selftext | score | num_comme | upvote_ratio | title_and_te | url | Season | created_time | created_hour |
| 2 | 1ajil9b | recruitinghel | 1707145431 | That was fas | I did not beli | 421 | 20 | 1 | That was fas | https://i.redd.it/p7eriedu7sgc1.jpeg | Feb | 2024-02-05T15:03:51 | 15 |
| 3 | 1auq54r | recruitinghel | 1708357012 | Social media | manager ÔÉ | 820 | 68 | 0.98 | Social media | https://i.redd.it/8e1d8jxhakjc1.jpeg | Feb | 2024-02-19T15:36:52 | 15 |
| 4 | 1avhpf6 | recruitinghel | 1708435903 | Red flag? | So I got hirec | 5 | 9 | 0.84 | Red flag? So | https://www.reddit.com/r/recruitinghell/cc | Feb | 2024-02-20T13:31:43 | 13 |
| 5 | 1gb1b3b | recruitinghel | 1729772490 | Why bother | I,ÄövÑv¥ve r | 163 | 50 | 0.99000001 | Why bother | https://i.redd.it/hz1fiuep5pwd1.jpeg | oct | 2024-10-24T12:21:30 | 12 |
| 6 | 1aytnkf | jobs | 1708778584 | I don,ÄövÑv | So I finished | 1 | 6 | 1 | I don,ÄövÑv | https://www.reddit.com/r/jobs/comments, | Feb | 2024-02-24T12:43:04 | 12 |
| 7 | 1g8uoek | jobs | 1729529931 | Is a former e | I saw our off | 1 | 13 | 0.33000001 | Is a former e | https://www.reddit.com/r/jobs/comments, | oct | 2024-10-21T16:58:51 | 16 |

Figure 4.11 Output CSV

## 5. Discussion

The results of this project show that the pipeline successfully processed all five weekly Reddit datasets and consistently produced four distinct outputs: clean records, filtered records, invalid rows, and detailed error logs. The consistent structure of the results across weeks demonstrates that the pipeline is reproducible, scalable. The Dataflow execution graphs also confirmed that each transform behaved as intended, and no stage failed during any of the five runs.

Within the Lifecycle & Pipeline module, there was a publication about designing pipeline with GCP and Dataflow for ETL, this guided me on my project. Also, Lifecycle principles such as raw-to-clean pathway, schema validation, quarantining problem rows, and maintaining logs directly shaped the storage structure and validation plan. Concepts from cloud computing guided the selection of managed services like Cloud Storage and Big data processing principles also influenced decisions on distributed transforms, autoscaling, and sharded output files.

Some improvements would be to automate multi-week ingestion through Cloud Scheduler and Pub/Sub triggers, enabling a true “hands-off” batch workflow. Another enhancement could involve archiving outputs to long-term storage or pushing the pipeline code to GitHub for preservation. Finally, adding lightweight notifications could help track pipeline completion over time.

I encountered some issues during the implementation. The most notable was the need to configure a VPC before Dataflow could initialize worker nodes. Without this network setup, early jobs failed due to inaccessible default network. Additionally, handling timestamp conversion from Unix values revealed irregularities in the dataset, leading to more quarantine entries than initially expected. These issues highlighted the importance of improved validation rules and failure-handling mechanisms in the pipelines. Dataflow API was not active by default, I needed series of authentications to communicate with the GCS bucket.

From a privacy perspective, the use of public Reddit data minimizes risk, but the project reinforces the need to handle user-generated content carefully, particularly when logs

preserve original text. Ensuring that data is not publicly exposed and restricting access through IAM controls helps maintain ethical and responsible data practices.

## 6. Conclusion

This project demonstrated how lifecycle-driven pipeline design can be implemented effectively in a cloud environment using Google Cloud Dataflow and Apache Beam. By structuring the workflow around raw ingestion, validation, filtering, and quarantining, the pipeline consistently processed multiple weeks of Reddit data and produced clean, organized outputs suitable for downstream analysis. The results highlight the value of applying lifecycle concepts to real-world, semi-structured datasets.

The key takeaway from this work is that well-designed pipelines not only automate repetitive data tasks but also improve reliability and data quality by enforcing structure at every stage of the lifecycle. If more time were available, I would extend the pipeline with automated scheduling and notifications, allowing weekly ingestion to run fully hands-off. Additional enhancements could include long-term archival strategies and broader monitoring tools to strengthen reproducibility and maintainability.

## 7. References


- Reddit Dataset Dump: https://academictorrents.com/details/1614740ac8c94505e4ecb9d88be8bed7b6afddd4
- Designing Data processing pipeline on GCP: https://medium.com/zeotap-customer-intelligence-unleashed/designing-data-processing-pipeline-on-google-cloud-platform-gcp-part-i-5a28644c5528
- Using Dataflow to Extract, Transform and Load Bike share data into Bigquery: https://bilalmkhan.medium.com/using-dataflow-to-extract-transform-and-load-bike-share-toronto-ridership-data-into-bigquery-e9b1dc199781


## 8. Appendix

- pipeline.py code

```
import argparse
import csv
import io
import json
from datetime import datetime

import apache_beam as beam
from apache_beam.options.pipeline_options import PipelineOptions,
SetupOptions
```

```python
# Expected Reddit CSV schema (order must match your files)
EXPECTED_COLUMNS = [
    "id",
    "subreddit",
    "created_utc",
    "title",
    "selftext",
    "score",
    "num_comments",
    "upvote_ratio",
    "title_and_text",
    "url",
    "Season"
]

# Extra columns for enriched data
VALID_COLUMNS = EXPECTED_COLUMNS + ["created_time", "created_hour"]
FILTERED_COLUMNS = VALID_COLUMNS
INVALID_COLUMNS = EXPECTED_COLUMNS + ["error_messages"]


def parse_csv_line(line):
    """Parse a CSV line into a dict using EXPECTED_COLUMNS."""
    # Skip header line by checking first column
    if line.startswith("id,subreddit,created_utc"):
        return None

    values = next(csv.reader([line]))
    if len(values) != len(EXPECTED_COLUMNS):
        # malformed row
        return {"__malformed__": True, "raw_line": line}

    return dict(zip(EXPECTED_COLUMNS, values))


def dict_to_csv_line(row, columns):
    """Convert a dict to a CSV line with the given column order."""
    output = io.StringIO()
    writer = csv.writer(output)
    writer.writerow([row.get(col, "") for col in columns])
    return output.getvalue().strip("\r\n")


class ValidateAndClassify(beam.DoFn):
    """
    Validate each row and classify into:
    - invalid (schema/type errors)
    - valid (meets business rules)
    - filtered (good structure but fails business rules)
    """

    def process(self, row):
        # Handle malformed rows from parse step
        if row is None:
            return

        if row.get("__malformed__"):
            row_copy = {"error_messages": "Malformed CSV row",
"raw_line": row["raw_line"]}
            yield beam.pvalue.TaggedOutput("invalid", row_copy)
            return
```

```
        errors = []

        # Required columns present and non-empty
        required = [
            "id",
            "subreddit",
            "created_utc",
            "title",
            "score",
            "num_comments",
            "upvote_ratio",
            "Season",
        ]
        for col in required:
            if row.get(col) in ("", None):
                errors.append(f"Missing value in {col}")

        # Type checks and conversions
        created_dt = None
        try:
            created_ts = int(row["created_utc"])
            created_dt = datetime.utcfromtimestamp(created_ts)
        except Exception:
            errors.append("created_utc must be valid integer
timestamp")

        try:
            score = int(row["score"])
        except Exception:
            errors.append("score must be int")
            score = None

        try:
            num_comments = int(row["num_comments"])
        except Exception:
            errors.append("num_comments must be int")
            num_comments = None

        try:
            ratio = float(row["upvote_ratio"])
            if not (0.0 <= ratio <= 1.0):
                errors.append("upvote_ratio must be between 0 and 1")
        except Exception:
            errors.append("upvote_ratio must be float")
            ratio = None

        # If any structural/type errors → invalid
        if errors or created_dt is None or num_comments is None:
            invalid_row = {col: row.get(col, "") for col in
EXPECTED_COLUMNS}
            invalid_row["error_messages"] = "; ".join(errors)
            yield beam.pvalue.TaggedOutput("invalid", invalid_row)
            return

        # Enrich with created_time and created_hour
        enriched = dict(row)
        enriched["created_time"] = created_dt.isoformat()
        enriched["created_hour"] = str(created_dt.hour)  # store as
string for CSV
```

```
        # Business rule: afternoon (12-17) AND >5 comments
        business_ok = 12 <= created_dt.hour <= 17 and num_comments >
5

        if business_ok:
            # Structurally valid + meets business rule
            yield beam.pvalue.TaggedOutput("valid", enriched)
        else:
            # Structurally valid, but does NOT meet business rules
            yield beam.pvalue.TaggedOutput("filtered", enriched)


def run(argv=None):
    parser = argparse.ArgumentParser()

    parser.add_argument("--input", required=True, help="GCS path to
input CSV")
    parser.add_argument("--output_clean", required=True, help="GCS
prefix for valid output CSV")
    parser.add_argument("--output_quarantine", required=True,
help="GCS prefix for invalid rows CSV")
    parser.add_argument("--output_filtered", required=True, help="GCS
prefix for filtered-out rows CSV")
    parser.add_argument("--log_output", required=True, help="GCS
prefix for JSON logs of invalid rows")

    known_args, pipeline_args = parser.parse_known_args(argv)

    # Make sure the pipeline can be pickled (good practice)
    pipeline_options = PipelineOptions(pipeline_args)
    pipeline_options.view_as(SetupOptions).save_main_session = True

    with beam.Pipeline(options=pipeline_options) as p:
        # Read lines from input CSV
        lines = p | "ReadCSV" >>
beam.io.ReadFromText(known_args.input)

        # Parse lines into dicts
        parsed = (
            lines
            | "ParseCSV" >> beam.Map(parse_csv_line)
            | "DropNone" >> beam.Filter(lambda r: r is not None)
        )

        # Validate & classify
        classified = (
            parsed
            | "ValidateAndClassify" >>
beam.ParDo(ValidateAndClassify())
              .with_outputs("valid", "invalid", "filtered")
        )

        valid_rows = classified.valid
        invalid_rows = classified.invalid
        filtered_rows = classified.filtered

        # ---- Write valid (clean) rows as CSV ----
        (
            valid_rows
            | "ValidToCSV" >> beam.Map(lambda row:
dict_to_csv_line(row, VALID_COLUMNS))
```

```
                | "WriteValid" >> beam.io.WriteToText(
                    known_args.output_clean,
                    file_name_suffix=".csv",
                    shard_name_template="-SSSSS-of-NNNNN",
                    header=",".join(VALID_COLUMNS),
                )
            )

            # ---- Write filtered-out rows as CSV ----
            (
                filtered_rows
                | "FilteredToCSV" >> beam.Map(lambda row:
dict_to_csv_line(row, FILTERED_COLUMNS))
                | "WriteFiltered" >> beam.io.WriteToText(
                    known_args.output_filtered,
                    file_name_suffix=".csv",
                    shard_name_template="-SSSSS-of-NNNNN",
                    header=",".join(FILTERED_COLUMNS),
                )
            )

            # ---- Write invalid (quarantine) rows as CSV ----
            (
                invalid_rows
                | "InvalidToCSV" >> beam.Map(lambda row:
dict_to_csv_line(row, INVALID_COLUMNS))
                | "WriteInvalid" >> beam.io.WriteToText(
                    known_args.output_quarantine,
                    file_name_suffix=".csv",
                    shard_name_template="-SSSSS-of-NNNNN",
                    header=",".join(INVALID_COLUMNS),
                )
            )

            # ---- Also write JSON logs for invalid rows ----
            (
                invalid_rows
                | "InvalidToJSON" >> beam.Map(lambda row:
json.dumps(row))
                | "WriteLogs" >> beam.io.WriteToText(
                    known_args.log_output,
                    file_name_suffix=".json",
                    shard_name_template="-SSSSS-of-NNNNN",
                )
            )


if __name__ == "__main__":
    run()
```

- requirements.txt

- IAM Account Permissions

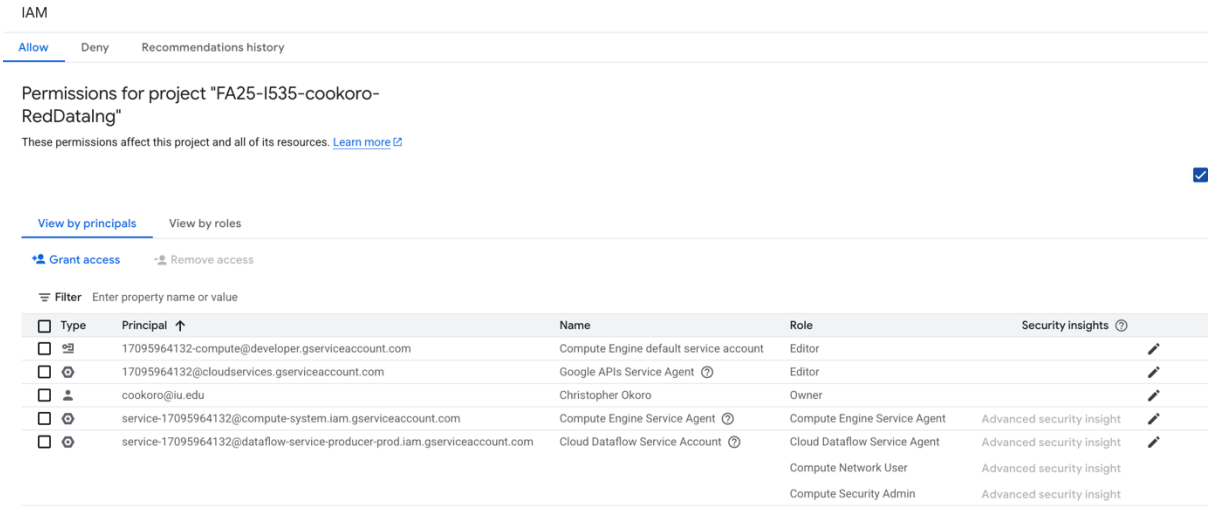



Figure 8.1 IAM roles for my user